JWT 게시판 구현 보고서 — ⚾ 야구 이야기 게시판

1 서비스 소개와 구현 범위

학번: [학번] 이름: [이름] 제출일: 2026-10-[일]

서비스 이름: 야구 이야기 게시판

기술 조합과 버전: React 19.3.0 (Vite 8.3.4) / Node.js Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2) / Node.js v22.18.0, Windows 11

GitHub 저장소 URL: [저장소 URL]

소스 코드 기준: main 브랜치, 코드 구현 기준 커밋 [커밋 SHA]

야구 이야기 게시판은 야구를 좋아하는 사람들이 직관 후기나 오늘 경기 감상, 응원하는 팀 이야기를 가볍게 남기는 공간입니다. 글은 로그인하지 않아도 누구나 읽을 수 있고, 로그인한 사용자만 글을 쓸 수 있습니다. 글의 수정과 삭제는 작성자 본인만 가능하며, 누가 작성자인지는 요청 본문이 아니라 서버가 JWT를 검증해서 결정합니다. 수업에서 다룬 Express 예제와 제공 템플릿이 모두 Express 기준이어서 React + Express 조합을 선택했고, 추가 설치 없이 PC에 있는 Node.js 22로 바로 실행할 수 있다는 점도 이유였습니다. 제공 템플릿에서는 인증 TODO가 있는 `backend/src/auth.js`, `frontend/src/api.js`와 화면 문구·오류 처리를 담당하는 `frontend/src/main.jsx`만 수정했고, CRUD·저장소·계정 검증 코드는 그대로 유지했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/src/app.js, store.js (제공 코드 유지)	토큰 없이 200 (T01)
로그인과 JWT 발급	완료	backend/src/auth.js `loadSecret`, `issueToken`	501 임시 응답 제거, 예시 키 거부
인증과 내 정보 조회	완료	backend/src/auth.js `currentUserId`, app.js GET /api/me	401을 3종 code로 구분
본인 글 작성·수정·삭제	완료	backend/src/auth.js `assertOwner`, app.js	타인 글 403, DB 불변
만료·로그아웃·오류 표시	완료	frontend/src/api.js, main.jsx	401은 로그아웃, 403은 유지
DB 데이터 유지	완료	backend/src/store.js (제공 코드 유지)	서버 재시작 후 유지 확인 (T15)

2 구조와 변경 내역



토큰은 `frontend/src/api.js`의 모듈 변수 `session`에만 보관합니다. localStorage를 쓰지 않으므로 새로고침하면 사라지고, 공개 조회(GET /api/posts)에는 헤더를 붙이지 않으며 /me·등록·수정·삭제 요청에만 `Authorization: Bearer <JWT>`를 붙입니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유	작성 출처
backend/src/auth.js `loadSecret`, `config`	없음 (새로 추가)	.env의 JWT_SECRET이 빈 값·예시 키·64자리 hex가 아니면 서버 시작 거부, 통과 시 32바이트 Buffer로 변환. iss/aud/TTL 설정 모음	BE-01 키 검증 요구사항	AI 초안 후 검토 반영
backend/src/auth.js `issueToken`	501 NOT_IMPLEMENTED 반환	`jwt.sign({sub: String(id)}, key, {HS256, expiresIn, issuer, audience})`로 발급	BE-01	AI 초안 후 검토 반영
backend/src/auth.js `currentUserId`	항상 데모 ID 1 반환	Bearer 헤더 파싱 → `jwt.verify` → exp·sub 형식 검사 → 실제 계정 확인. 실패를 TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED로 구분	BE-02	AI 초안 후 검토 반영

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유	작성 출처
backend/src/auth.js `assertOwner`	빈 함수 (검사 없음)	사용자 ID와 글의 authorId가 다르면 403 FORBIDDEN	BE-03	AI 초안 후 검토 반영
backend/src/app.js	서버 시작 콘솔 문구	"인증 TODO 미완성" → "JWT 인증 적용" 문구만 변경	미완성 표시 제거	본인 작성
frontend/src/api.js `authorizationHeaders`, `request`, `login`, `canEdit`, `MESSAGES`	빈 헤더, 로그인 예외 던짐, canEdit 항상 true	토큰 있을 때만 Bearer 헤더, 401이면 세션 해제, code별 안내 문구, 로그인 응답을 메모리에 저장, 작성자 비교	FE-01~04	AI 초안 후 검토 반영
frontend/src/main.jsx `run`, 화면 문구	데모 배너, 오류는 메시지만 표시	404면 목록 재조회, 401이면 편집 상태 정리, 비로그인 시 등록 버튼 비활성화와 안내, 타인 글 안내 문구, 야구 주제 제목·배너	FE-03·04, 주제 반영	AI 초안 후 검토 반영
frontend/index.html	페이지 제목	"야구 이야기	JWT 게시판"	주제 반영
tools/manual_tests.sh, backend/scripts/check_claims.mjs	없음 (보조 파일 추가)	T01~T17 curl 자동 기록 스크립트, 잘못된 클레임 토큰 거부 확인 스크립트	테스트 재현성	AI 초안 후 검토 반영

users.js, store.js, errors.js, style.css, vite.config.js, package.json은 제공 코드를 그대로 유지했습니다.

3 핵심 코드 설명

3.1 로그인과 JWT 발급

파일 / 함수: `backend/src/auth.js` — `loadSecret` (9행), ~~`issueToken` (33행)~~ / `backend/src/app.js` — `POST /api/auth/login` (45행~)

```
// auth.js
function loadSecret() {
  const raw = (process.env.JWT_SECRET || "").trim();
  if (!raw) throw new Error("JWT_SECRET이 비어 있습니다. .env를 확인하세요.");
  if (raw === EXAMPLE_SECRET) throw new Error("JWT_SECRET이 예시 키 그대로입니다. 새 난수로 바꾸세
요.");
  if (!/^[0-9a-fA-F]{64}$/.test(raw)) throw new Error("JWT_SECRET은 64자리 16진수 문자열이어야 합니
다.");
  return Buffer.from(raw, "hex"); // 64자리 hex → 32바이트 키
}
export function issueToken(user) {
  return jwt.sign({ sub: String(user.id) }, config.secret, {
    algorithm: "HS256", expiresIn: config.ttl, issuer: config.issuer, audience: config.audience,
  });
}
// app.js
const input = fields(req.body, { email: 254, password: 128 });   // 형식 오류 → 400 BAD_INPUT
const user = verifyCredentials(input.email, input.password);     // 제공 코드 유지
if (!user) throw new ApiError(401, "INVALID_CREDENTIALS", "...");
const accessToken = issueToken(user);
res.json({ accessToken, expiresIn: Number(process.env.TOKEN_TTL || 90), user });
```

입력은 JSON의 email·password입니다. 제공 코드의 `fields`가 두 필드가 문자열이고 길이 범위 안인지 확인하고, 아니면 400 BAD_INPUT을 던집니다. `verifyCredentials`는 scrypt 해시를 비교해서 계정이 맞을 때만 공개 정보(id, name, email)를 돌려주고, 틀리면 null이라 401 INVALID_CREDENTIALS가 됩니다. 계정이 확인된 경우에만 `issueToken`이 호출됩니다. 토큰 payload에는 사용자 ID를 문자열로 바꾼 `sub`만 직접 넣고, `exp`·`iss`·`aud`는 옵션으로 라이브러리가 채웁니다. 비밀번호나 해시는 토큰에도 응답에도 들어가지 않습니다. 서명 키는 서버 시작 시 `loadSecret`이 검증한 32바이트 Buffer입니다. 실제로 예시 키·빈 키·`abc123`으로 시작해 봤을 때 각각 다른 메시지로 서버가 멈췄습니다. 응답은 200과 `accessToken`, `expiresIn`(90), `user`입니다.

3.2 브라우저 토큰 저장과 전송

파일 / 함수: `frontend/src/api.js` — `session` (4행), `authorizationHeaders` (12행), ~~`request` (28행)~~, `login` (56행~)

```
let session = { accessToken: "", user: null };          // 브라우저 메모리에만 보관
function authorizationHeaders() {
  return session.accessToken ? { Authorization: `Bearer ${session.accessToken}` } : {};
}
export async function request(path, options = {}) {
  const response = await fetch(`/api${path}`, { ...options,
    headers: { "Content-Type": "application/json", ...authorizationHeaders(), ...options.headers
} });
  const data = response.status === 204 ? null : await response.json();   // 204는 본문 없음
  if (!response.ok) {
    if (response.status === 401) clearSession();      // 401: 토큰·사용자 해제. 403은 유지
    const code = data?.code;
    const error = new Error(MESSAGES[code] || data?.message || code || `HTTP ${response.status}
`);
    error.status = response.status; error.code = code; throw error;
  }
  return data;
}
export async function login(email, password) {
  const data = await request("/auth/login", { method: "POST", body: JSON.stringify({ email, pass
word }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

로그인 폼의 값은 `login`이 JSON으로 POST합니다. 성공하면 응답의 `accessToken`과 `user`를 모듈 변수 `session`에 넣고 `main.jsx`가 `getSession().user`로 화면의 "Alice님 로그인 중"을 갱신합니다. 실패(401 INVALID_CREDENTIALS)는 `request`가 예외를 던지므로 `session`은 바뀌지 않고 화면에는 "이메일 또는 비밀번호가 틀렸습니다"만 표시됩니다. `authorizationHeaders`는 토큰이 있을 때만 헤더 객체를 반환해서 토큰이 없을 때 `Bearer undefined` 같은 값이 생기지 않습니다. 로그아웃 버튼은 `clearSession()`으로 변수를 비우고 화면 상태를 지웁니다. localStorage를 쓰지 않아 새로고침하면 로그인이 풀리는데, 이것은 과제에서 요구한 정상 동작입니다. `request`의 401 처리로 토큰 만료나 변조 응답을 받으면 자동으로 로그아웃되고 code에 맞는 안내가 뜹니다. 403은 로그인이 유효한 상태이므로 세션을 지우지 않습니다.

3.3 서버 JWT 검증

파일 / 함수: `backend/src/auth.js` — `currentUserId` (52행~)

```
export function currentUserId(req) {
  const header = req.get("Authorization") || "";
  const parts = header.trim().split(/\s+/);
  if (parts.length !== 2 || parts[0].toLowerCase() !== "bearer" || !parts[1])
    throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다. ...");
  let payload;
  try {
    payload = jwt.verify(parts[1], config.secret, {
      algorithms: ["HS256"], issuer: config.issuer, audience: config.audience });
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError)
      throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
    throw new ApiError(401, "INVALID_TOKEN", "토큰이 유효하지 않습니다.");
  }
  if (typeof payload.exp !== "number") throw new ApiError(401, "INVALID_TOKEN", "토큰에 만료 시각
이 없습니다.");
  if (typeof payload.sub !== "string" || !/^[1-9]\d*$/.test(payload.sub))
    throw new ApiError(401, "INVALID_TOKEN", "토큰의 사용자 정보가 올바르지 않습니다.");
  const user = findPublic(Number(payload.sub));
  if (!user) throw new ApiError(401, "INVALID_TOKEN", "존재하지 않는 사용자입니다.");
  return user.id;
}
```

보호 API(/me, POST·PUT·DELETE /api/posts)는 라우트 첫 줄에서 이 함수를 호출합니다. 입력은 Authorization 헤더 하나입니다. 처리 순서는 (1) 헤더가 정확히 `Bearer <토큰>` 두 조각인지, (2) `jwt.verify`로 서명이 내 키로 만든 것이 맞는지, 알고리즘이 HS256인지, exp가 지나지 않았는지, iss·aud가 설정값과 같은지, (3) 라이브러리가 검사하지 않는 exp 존재 여부와 sub 형식, (4) sub에 해당하는 계정이 실제로 있는지입니다. 출력은 검증된 사용자 ID 숫자 하나뿐이고, 토큰의 다른 내용은 라우트에 넘기지 않습니다. 실패는 모두 HTTP 401이지만 code가 다릅니다. 헤더 자체가 없거나 형식이 틀리면 TOKEN_REQUIRED, 서명이 정상인데 시간이 지났으면 TOKEN_EXPIRED(`TokenExpiredError`일 때만), 그 밖의 모든 경우(서명 불일치, alg 불일치, iss/aud 불일치, exp·sub 누락, 없는 사용자)는 INVALID_TOKEN입니다. `app.js`의 공통 에러 핸들러가 status 401이면 `WWW-Authenticate: Bearer` 헤더를 붙입니다. `jwt.decode`를 쓰지 않은 이유는 decode는 base64를 풀어 읽기만 하고 서명을 확인하지 않아서 누구나 sub를 2로 고친 토큰을 만들 수 있기 때문입니다. 실제로 payload의 sub만 2로 바꾼 토큰을 보내 보면 서명이 맞지 않아 INVALID_TOKEN이 됩니다(docs/evidence/check_claims.log).

3.4 작성자 권한 검사

파일 / 함수: `backend/src/auth.js` — `assertOwner` (103행) / ~~`backend/src/app.js` — PUT·DELETE /api/posts/:id (77행)~~, POST /api/posts (69행~)

```
// auth.js
export function assertOwner(userId, authorId) {
  if (userId !== authorId)
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
}
// app.js (제공 코드 유지)
app.post("/api/posts", (req, res) => {
  const userId = currentUserId(req);                        // 작성자는 토큰에서 결정
  const p = store.create(fields(req.body, { title: 100, content: 5000 }), userId);
  res.status(201).location(`/api/posts/${p.id}`).json(p);
});
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req), id = postId(req), p = getPost(id);   // 인증 → 존재(404)
  assertOwner(userId, p.authorId);                                        // 권한(403)
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));  // 입력(400) → 변
경
});
```

비교 대상은 토큰에서 검증한 사용자 ID와 DB에 저장된 글의 `authorId`입니다. 요청 본문은 비교에 쓰지 않습니다. 등록할 때 작성자는 `currentUserId`가 돌려준 ID를 `store.create`에 넘겨 서버가 정하고, 본문에 `authorId`나 `userId`가 들어 있으면 제공 코드의 `fields`가 허용되지 않은 필드로 보고 400 BAD_INPUT을 냅니다. 그래서 Bob이 Alice인 척 글을 만들 수 없습니다(T12). 401과 403의 차이는 "누구인지 모르는 것"과 "누구인지는 알지만 권한이 없는 것"입니다. 토큰이 없으면 `currentUserId`에서 401로 끝나고 글 존재 확인까지 가지 않으므로, 토큰 없이 없는 글을 수정하면 404가 아니라 401 TOKEN_REQUIRED가 됩니다. 정상 토큰이면 `getPost`에서 404가 먼저 나고, 글이 있으면 `assertOwner`가 403을 냅니다. 입력 검증은 권한 검사 뒤에 있어서 Bob이 Alice 글에 공백 제목을 보내도 400이 아니라 403입니다(T07-x). 403으로 거부되면 `store.update`에 도달하지 않으므로 DB는 그대로이고, 재조회로 원문이 유지됨을 확인했습니다.

4 테스트 결과와 증거

실행 환경: Windows 11 / Node.js v22.18.0 / curl (Git Bash), Python 3.14 (check_api.py) / 백엔드 `npm start`, 프론트 `npm run build` → `npm run preview`

로컬 배포 확인: `npm run build` 성공 로그 `docs/evidence/frontend_build.log`. `npm run preview`로 4173에 접속해 페이지 제목과 `/api/health` 프록시 응답 200을 확인. 브라우저 화면 캡처는 `docs/evidence/ui-*.png`.

테스트 일시 및 코드 버전: 2026-10-28, 커밋 [커밋 SHA]

토큰 TTL: 일반 테스트 90초 / 만료 테스트 3초 (`TOKEN_TTL=3`으로 별도 실행)

API 테스트는 curl로 수행했고 `tools/manual_tests.sh`로 재현할 수 있습니다. 로그의 토큰은 `<TOKEN(masked)>`로 가렸습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	토큰 없이 GET /api/posts, GET /api/posts/5	200	200 / 해당 없음 (두 요청 모두)	PASS	manual_tests_T01-T17.log T01-a, T01-b

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T02	Alice 로그인 POST /api/auth/login	200, 토큰·user, 화면 로그인 표시	200 / 해당 없음. accessToken 3조각, 헤더 alg HS256, payload sub "1"·iss·aud·exp	PASS	T02, 그림 1
T03	틀린 비밀번호 / 없는 이메일 로그인	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS (둘 다), password 누락은 400 / BAD_INPUT	PASS	T03-a, b, c
T04	토큰 없이 GET /me, POST /posts (+ `Token abc`, `Bearer not-a-jwt`)	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED (둘 다), WWW-Authenticate: Bearer. 잘못된 스킴은 TOKEN_REQUIRED, Bearer 뒤 비JWT는 INVALID_TOKEN	PASS	T04-a~d
T05	Alice 토큰으로 POST /posts, GET /me	201 authorId 1, /me는 Alice	201 / Location: /api/posts/5, authorId 1 · 200 / id 1 Alice	PASS	T05-a, b
T06	Alice 토큰으로 본인 글 PUT, DELETE	200, 204, DB 반영	200 / 제목 변경 ·authorId·createdAt 유지 · 204 / 본문 없음 · 삭제 글 재조회 404	PASS	T06-a~e
T07	Bob 토큰으로 Alice 글(5) PUT	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN. 공백 제목이어도 403. 재조회 원문 동일	PASS	T07, T07-x
T08	Bob 토큰으로 Alice 글(5) DELETE	403 FORBIDDEN, 글 유지	403 / FORBIDDEN. 재조회 원문 동일	PASS	T08
T09	Alice 토큰 서명 첫 글자 변조 → GET /me, POST, PUT, DELETE	401 INVALID_TOKEN, 데이터 불변	401 / INVALID_TOKEN (4건 모두), WWW-Authenticate: Bearer. 재조회 원문 동일, 글 수 1	PASS	T09-a~d
T10	TOKEN_TTL=3 서버에서 발급 → 4초 뒤 GET /me, POST, PUT, DELETE	401 TOKEN_EXPIRED, 화면 로그아웃 안내	401 / TOKEN_EXPIRED (4건 모두), WWW-Authenticate: Bearer. 자동 검사 --expiry 48/48 PASS. 글 재조회 불변	PASS	manual_tests_T10_expiry.log, 그림 7
T11	로그아웃 후 GET /me (헤더 없음)	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	T11, 그림 5
T12	Bob 토큰으로 POST /posts에 authorId:1 / userId:1 추가	400 BAD_INPUT, 미생성	400 / BAD_INPUT (둘 다). 목록에 위조 글 없음	PASS	T12-a, b

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T13	Alice 토큰으로 빈 제목 / 공백 본문 / 제목 101자 / 본인 글에 공백 제목 PUT	400 BAD_INPUT, DB 불변	400 / BAD_INPUT (4건 모두). 재조회 원문 동일	PASS	T13-a~d
T14	GET·PUT·DELETE /posts/999999 (정상 토큰), GET /posts/abc	404 NOT_FOUND, 400 BAD_INPUT	404 / NOT_FOUND (3건) · 400 / BAD_INPUT. 토큰 없이 없는 글 PUT은 401 / TOKEN_REQUIRED	PASS	T14-a~e
T15	글 작성 후 서버 프로세스 종료·재실행	글·작성자·내용 유지	종료 후 health 연결 실패 확인 → 재실행 → 목록 JSON이 종료 전과 완전히 동일	PASS	manual_tests_T15_restart.log
T16	로그인 후 새로고침	글 유지, 로그인 해제	화면 확인 (HTTP 오류 없음). F5 후 글 목록 2개 그대로 유지, 로그인 표시는 "Bob님 로그인 중" → "로그아웃 상태"로 해제	PASS	그림 8
T17	토큰 없이 Alice 글 PUT·DELETE / Bob 본인 글 POST·PUT·DELETE	무토큰 401, Bob 본인 성공	401 / TOKEN_REQUIRED (2건) · 201 / Location, 200, 204 · 삭제 후 404	PASS	T17-a~f

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	200	200 (T01, T02, T05-b, T06-a, T17-d)	manual_tests_T01-T17.log
새 글 등록	201 및 Location	201, `Location: /api/posts/5` (T05-a), `/api/posts/7` (T17-c)	같음
본인 글 삭제	204 및 빈 본문	204 No Content, 본문·Content-Length 없음 (T06-d, T17-e)	같음
입력 오류·추가 작성자 필드	400 BAD_INPUT	400 / BAD_INPUT (T12 ×2, T13 ×4, T14-d, T03-c)	같음
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS (T03-a, b)	같음
토큰 없음·헤더 형식 오류	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED + WWW-Authenticate: Bearer (T04-a, b, c, T11, T14-e, T17-a, b)	같음
변조·클레임 오류	401 INVALID_TOKEN	401 / INVALID_TOKEN + WWW-Authenticate: Bearer (T04-d, T09 ×4, check_claims 9건)	같음, check_claims.log

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED + WWW-Authenticate: Bearer (/me, POST, PUT, DELETE)	manual_tests_T10_expiry.log
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN (T07, T07-x, T08)	manual_tests_T01-T17.log
없는 글	404 NOT_FOUND	404 / NOT_FOUND (T06-e, T14-a, b, c, T17-f)	같음

WWW-Authenticate 확인: 보호 API의 401 응답(TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED) 전부에 `WWW-Authenticate: Bearer` 헤더가 있었고, 로그인 실패 401(INVALID_CREDENTIALS)에는 없습니다. `app.js`의 공통 에러 핸들러가 status 401일 때 헤더를 붙이는데, 로그인 401에도 붙지만 명세는 보호 API에서만 요구하므로 문제 없습니다. 자동 검사 `check_api.py`도 이 헤더를 확인하며 통과했습니다.

401과 403의 화면 차이: 401(만료·변조·토큰 없음)을 받으면 `api.js`의 `request`가 `clearSession()`을 호출해 로그인 표시가 "로그아웃 상태"로 바뀌고 "로그인 시간이 만료되었습니다. 다시 로그인하세요" 같은 code별 안내가 뜹니다. 403은 세션을 지우지 않아 "Alice님 로그인 중" 표시가 유지된 채 "본인이 작성한 글만 수정·삭제할 수 있습니다"만 표시됩니다. UI에서는 타인 글에 버튼이 없으므로 403은 curl로 직접 확인했습니다.

실패 뒤 데이터 불변: T07, T08, T09, T13 각각의 실패 응답 뒤 `GET /api/posts/5`로 재조회한 JSON이 실패 전과 바이트 단위로 동일했고(스크립트가 문자열 비교로 확인), 목록 개수도 1개로 변함이 없었습니다. T12의 위조 등록 뒤 목록에도 새 글이 없었습니다.

토큰 없이 없는 글 수정 / 정상 토큰으로 없는 글 수정: `PUT /api/posts/999999`를 토큰 없이 보내면 401 TOKEN_REQUIRED(T14-e), Alice 토큰으로 보내면 404 NOT_FOUND(T14-b)였습니다. 인증이 글 존재 확인보다 먼저 실행되기 때문입니다.

추가 클레임 검사 (`backend/scripts/check_claims.mjs`, 로컬 실습 키로 직접 서명): iss 불일치, aud 불일치, exp 누락, sub 누락, sub 숫자형, 없는 사용자(sub 99), alg=none, HS512 서명, payload만 변조 — 9건 모두 401 INVALID_TOKEN, 정상 토큰 대조군은 200. 단순 payload 변조와 달리 iss·aud·exp·sub 케이스는 서명이 올바른 토큰이므로 클레임 검사가 실제로 동작함을 보여줍니다.

증거 1: 로그인 성공과 토큰이 첨부된 요청 (T02, T05)

그림 1. Alice 로그인 (`ui-02-login.png`): 화면에 "Alice님 로그인 중", Network의 `POST /api/auth/login` 200 OK

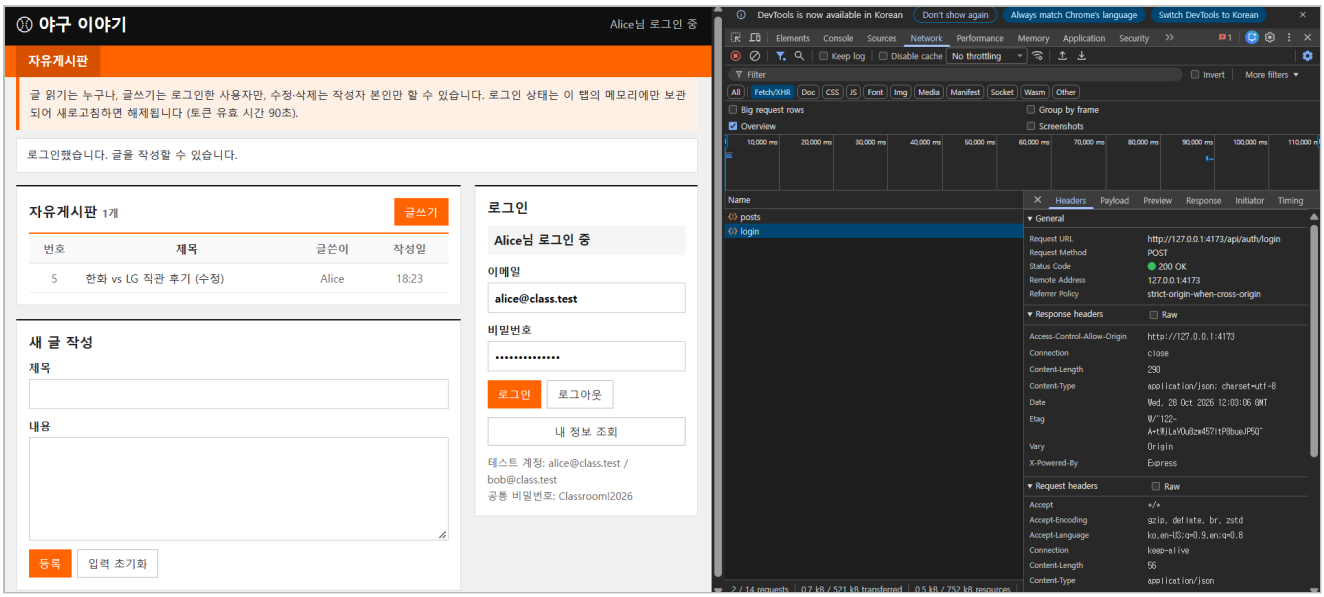


그림 2. 글 등록 (`ui-05-network1.png`): `POST /api/posts` 201 Created, 응답 헤더 `Location: /api/posts/8`

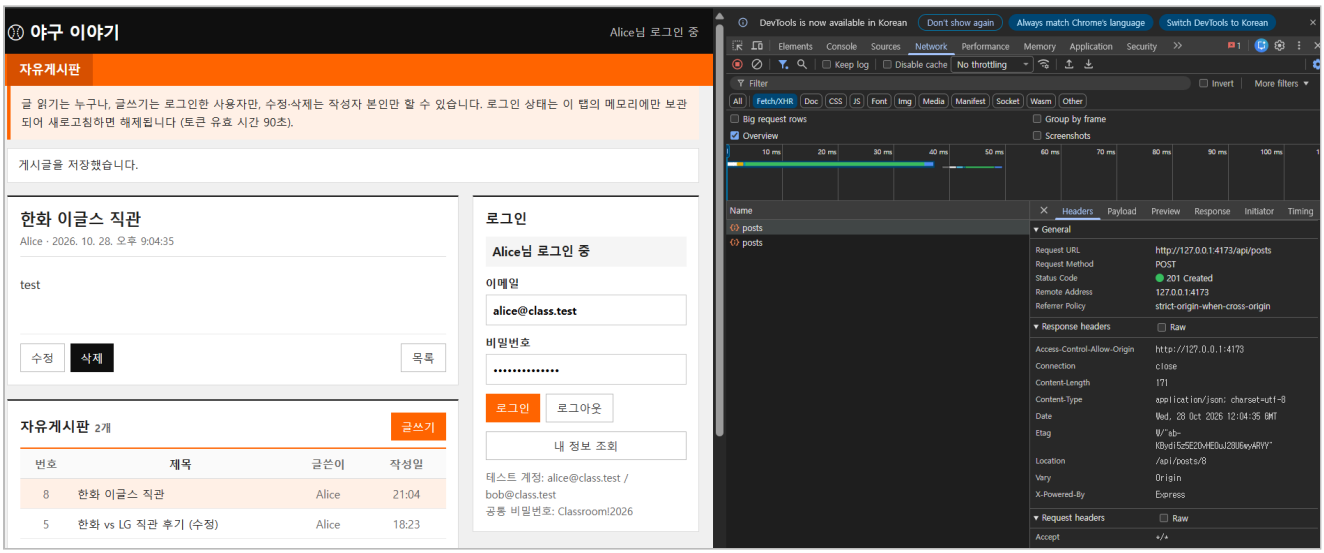


그림 3. 같은 POST 요청의 Request Headers (`ui-05-network2.png`): `Authorization` 헤더가 전송됨 (값 전체를 가림)

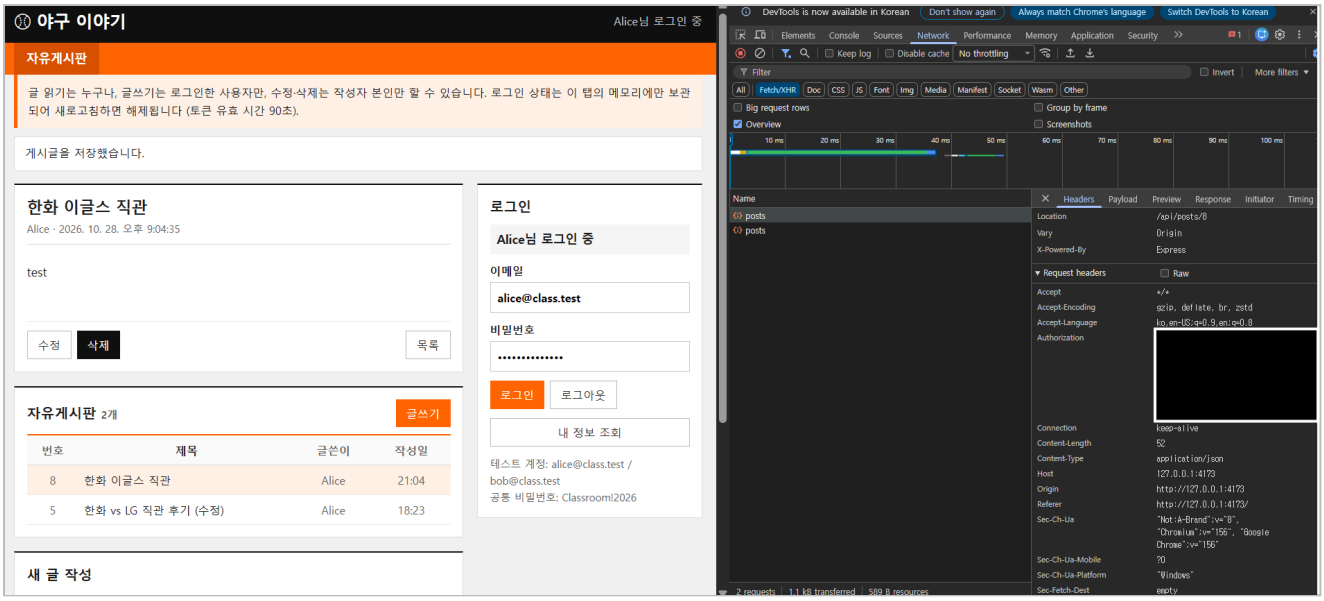


그림 4. 로그인 상태의 다른 요청 Request Headers (`ui-05-network3.png`): `Authorization: Bearer eyJhbGciOiJIUzI1NiIs…` 형식 확인 (앞부분만 남기고 가림)

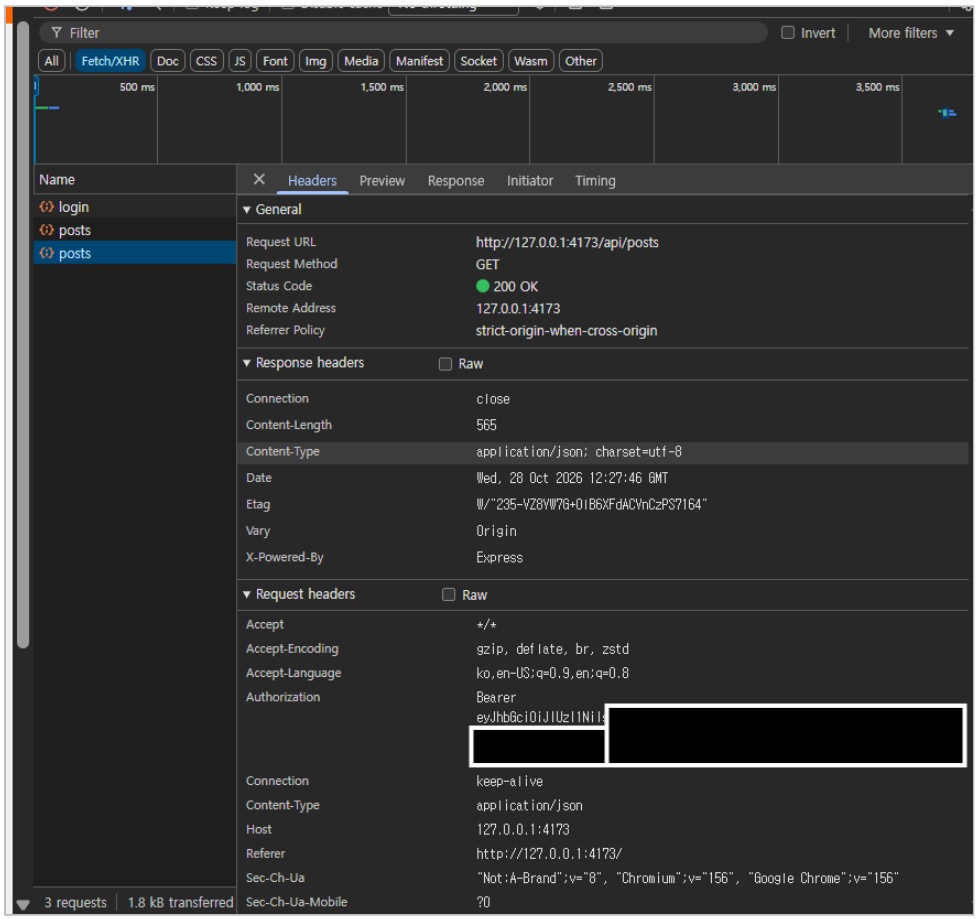
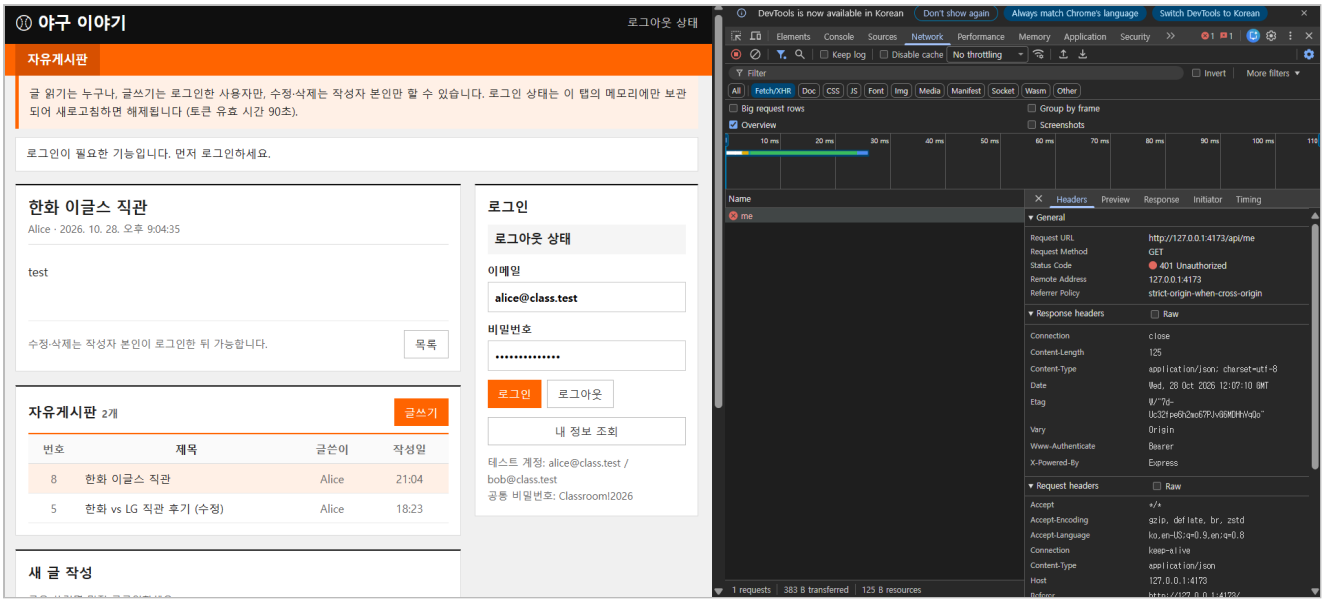
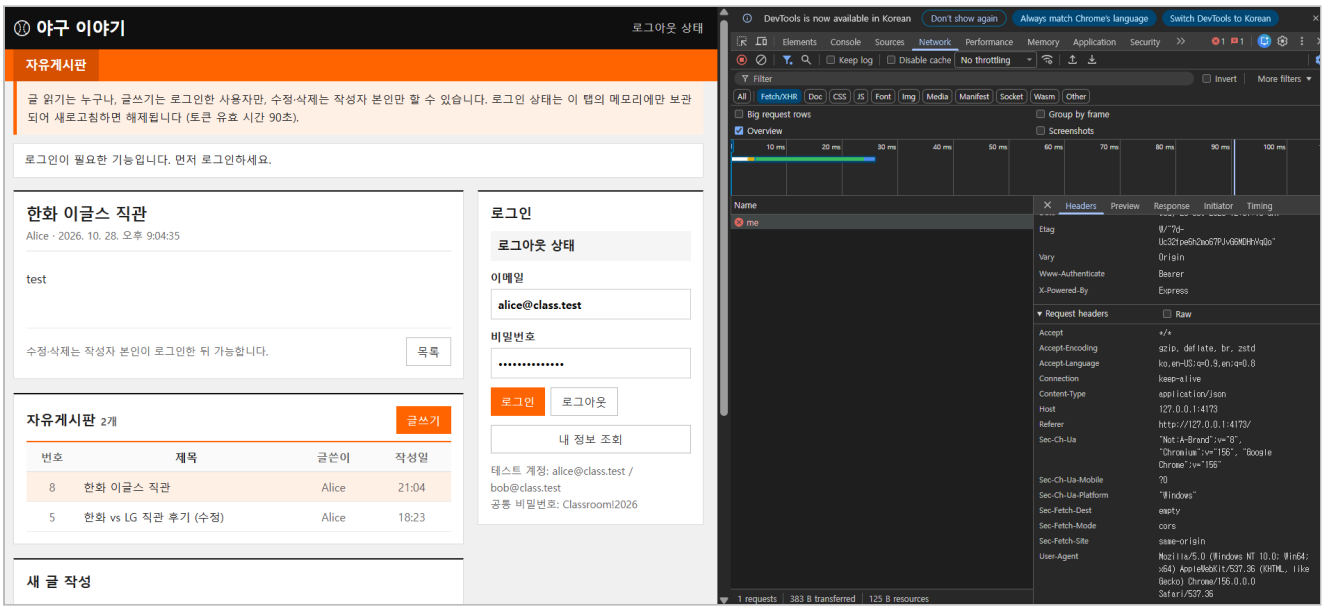


그림 5. 로그아웃 뒤 내 정보 조회 (`ui-11-logout1.png`, `ui-11-logout2.png`, T11): `GET /api/me` 401, 응답 헤더 `WWW-Authenticate: Bearer`, Request Headers에 `Authorization` 없음, 화면은 "로그아웃 상태"와 "로그인이 필요한 기능입니다" 안내

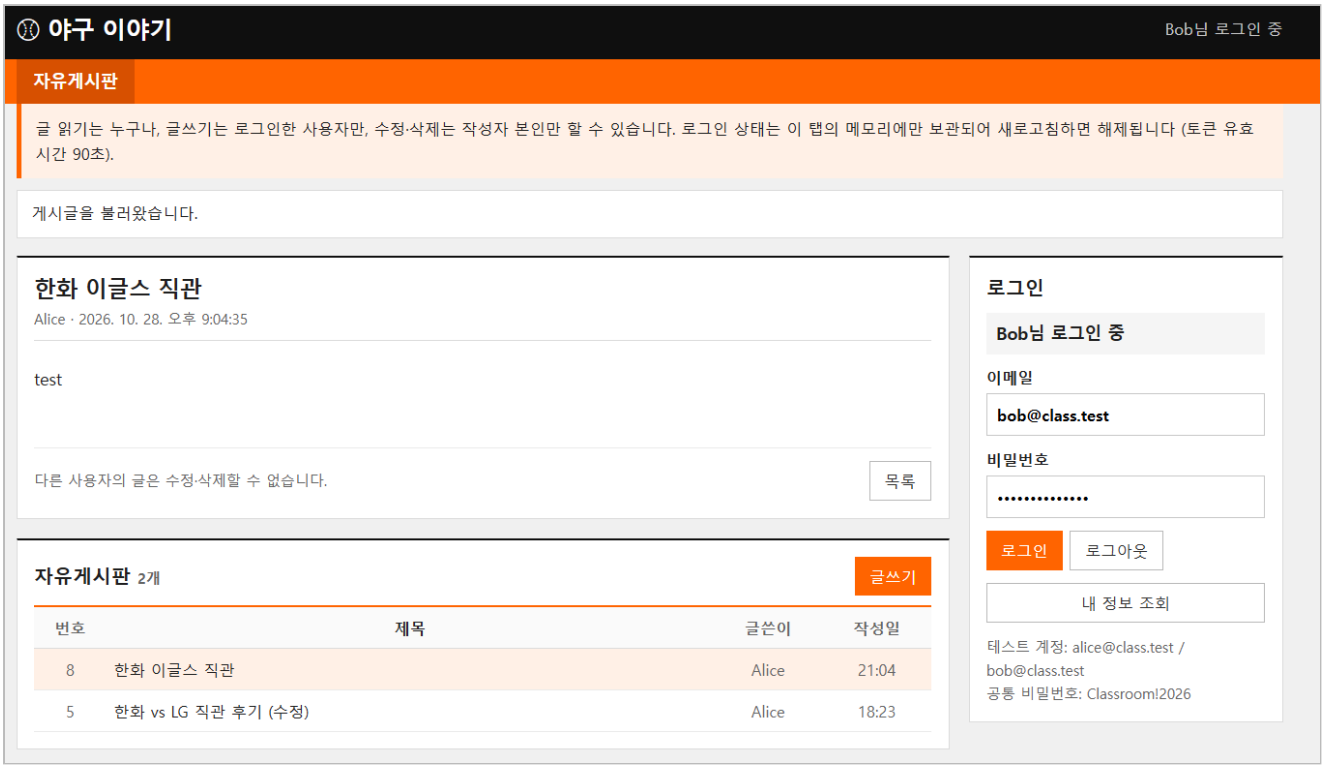




증거 2: Bob이 Alice의 글 수정·삭제를 시도한 서버 응답 (T07, T08)

서버 응답은 `manual_tests_T01-T17.log`의 T07·T08 블록(403 FORBIDDEN, 재조회 시 원문 유지)에 있습니다. 화면에서는 Bob으로 로그인한 상태에서 Alice 글을 열면 수정·삭제 버튼이 없고 안내 문구만 보입니다.

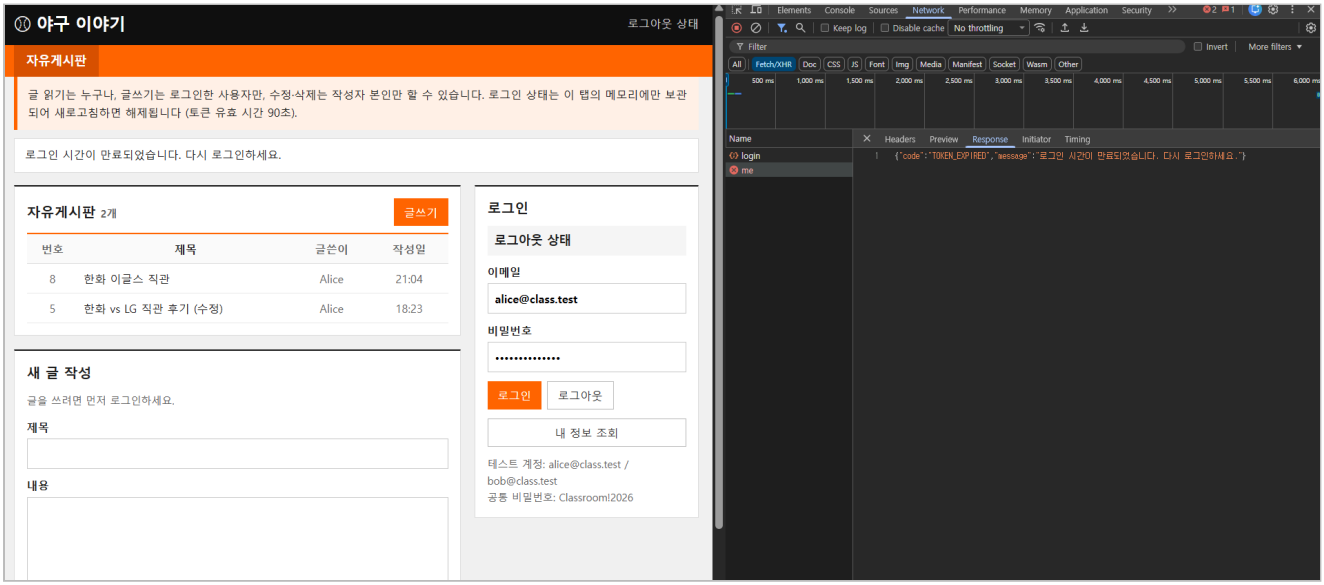
그림 6. Bob이 Alice 글을 연 화면 (`ui-07-other-post.png`): "Bob님 로그인 중" 유지, "다른 사용자의 글은 수정·삭제할 수 없습니다."



증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리 (T09, T10)

변조 토큰 응답은 `manual_tests_T01-T17.log`의 T09, 만료 응답 4건은 `manual_tests_T10_expiry.log`에 있습니다.

그림 7. 만료 (`ui-10-expired.png`, `TOKEN_TTL=3` 서버에서 로그인 후 5초 이상 지나 내 정보 조회): `GET /api/me` 401, 응답 `{"code":"TOKEN_EXPIRED",…}`, 화면은 "로그아웃 상태"와 "로그인 시간이 만료되었습니다. 다시 로그인하세요."



그 밖의 화면 증거 (T16, 비로그인)

그림 8. 새로고침 (`ui-16-refresh.png`, T16): Bob 로그인 상태에서 F5 후 글 목록은 그대로, 로그인 표시는 "로그아웃 상태"

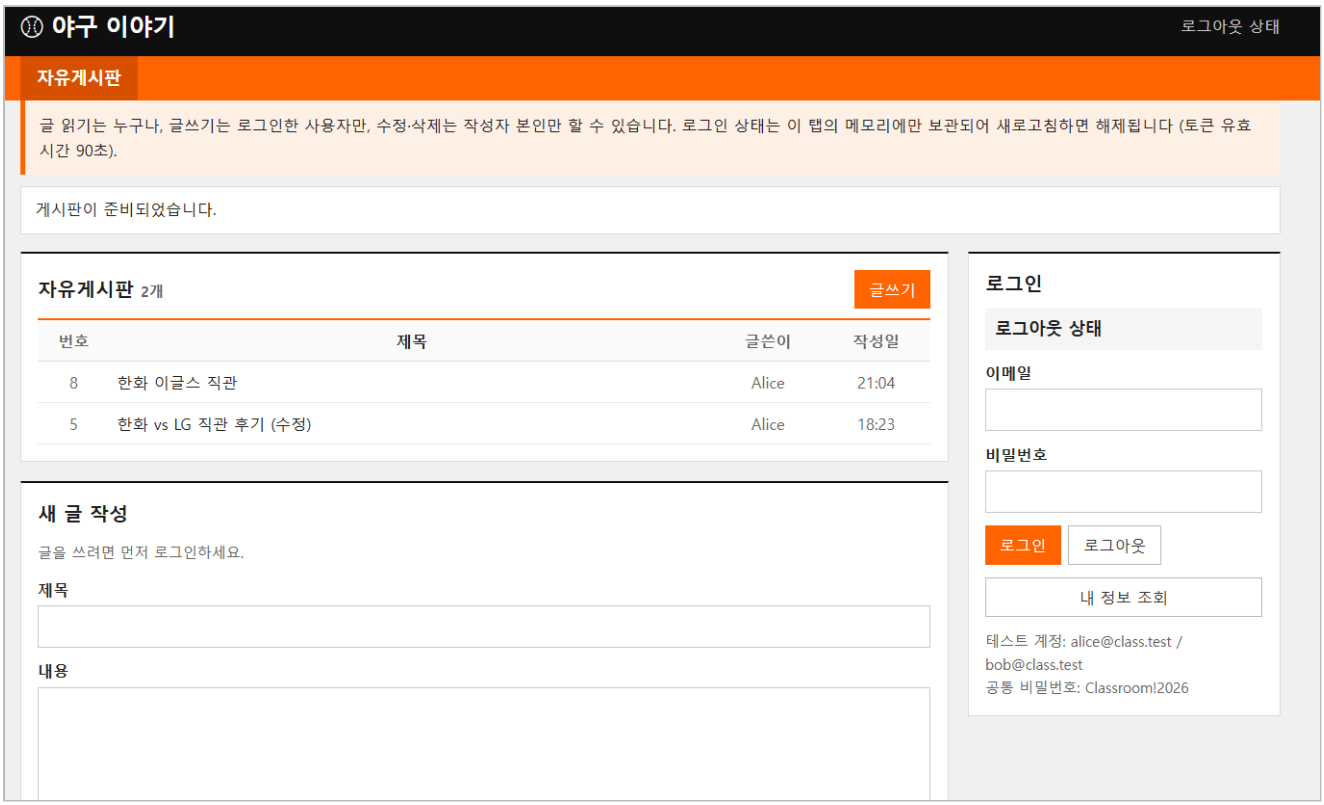
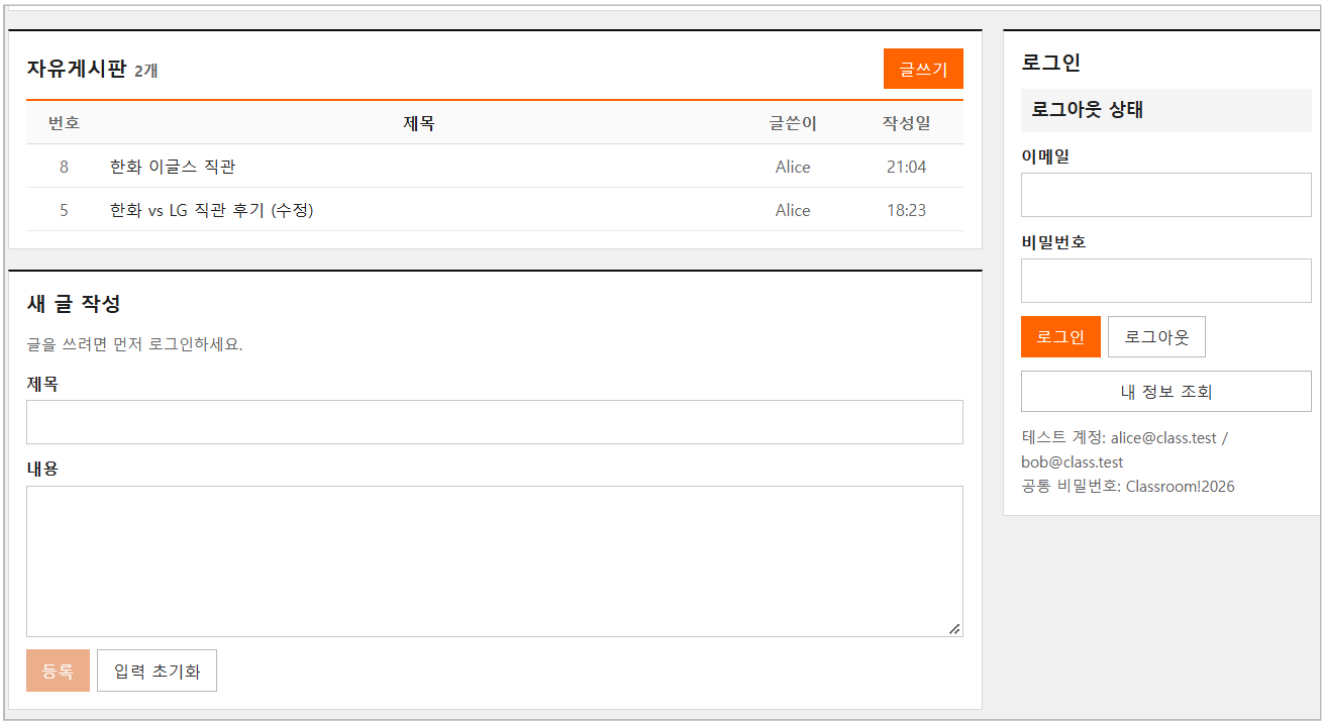


그림 9. 비로그인 첫 화면 (`ui-01-public.png`): "글을 쓰려면 먼저 로그인하세요." 안내와 비활성 등록 버튼



자동 검사: `py -I tools/check_api.py --mode final` 40/40 PASS(`check_api_final.log`), `--expiry` 48/48 PASS. 이 스크립트는 교수님이 제공한 것으로, 명세의 상태 코드·code·WWW-Authenticate·Location·데이

터 불변을 요청으로 확인합니다. UI 동작과 서버 재시작은 별도로 수동 수행했습니다.

5 AI 활용과 참고 자료

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude Code (Claude Fable 5.1)	설계: 과제 문서 분석, 채점 포인트·작업 순서 정리	작업 계획, docs/ai-log.md	참고
Claude Code (Claude Fable 5.1)	생성: BE-01~~03, FE-01~~04 코드 초안	backend/src/auth.js, frontend/src/api.js, main.jsx	검토 후 그대로 반영
Claude Code (Claude Fable 5.1)	테스트: curl 테스트 스크립트, 클레임 검사 스크립트 작성과 실행	tools/manual_tests.sh, backend/scripts/check_claims.mjs, docs/evidence/	검토 후 그대로 반영
Claude Code (Claude Fable 5.1)	문장 정리: README·보고서 초안	README.md, docs/report.md	수정 후 반영

대표 사례 1 — 서버 JWT 검증(BE-02)

도구 / 날짜: Claude Code / 2026-10-28

요청한 내용: "BE-02 토큰 검증과 /me, BE-03 작성자 권한을 명세대로 구현. 401을 TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED로 구분하고 decode가 아니라 verify를 쓸 것"

AI가 제안한 내용: `jwt.verify`에 `algorithms: ["HS256"]`, `issuer`, `audience`를 넘겨 서명·알고리즘·만료·iss·aud를 한 번에 검사하고, `TokenExpiredError`만 TOKEN_EXPIRED로 분리. 라이브러리가 exp 없는 토큰은 통과시키므로 `payload.exp` 타입을 직접 검사하고, sub 형식 검사 후 `findPublic`으로 실제 계정 존재까지 확인하라는 제안.

실제 반영 위치: backend/src/auth.js `currentUserId`

내가 확인하고 수정한 내용: "exp 없는 토큰이 verify를 통과한다"는 설명이 맞는지 의심스러워서 그대로 믿지 않고 검증 스크립트(check_claims.mjs)로 exp 없는 토큰을 직접 만들어 보냈습니다. 서버의 직접 검사 덕분에 401이 나왔고, 이 검사 줄을 빼면 통과한다는 점도 확인했습니다. 코드는 수정 없이 반영했습니다.

검증 방법과 결과: check_api.py final 40/40, T04·T09·T10·T14, check_claims 10/10. 특히 alg=none 토큰이 거부되는 것은 `algorithms` 옵션을 명시했기 때문입니다.

대표 사례 2 — 프론트 401/403 처리(FE-03)

도구 / 날짜: Claude Code / 2026-10-28

요청한 내용: "401이면 로그인 상태 해제와 재로그인 안내, 403은 유지, 404는 목록 갱신, 204는 JSON 파싱 금지"

AI가 제안한 내용: `request` 한 곳에서 `response.status === 401`일 때 `clearSession()`을 호출하고, code별 문구 표 `MESSAGES`로 INVALID_CREDENTIALS·TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED를 구분. 404는 `main.jsx`의 `run`에서 상세를 닫고 목록을 재조회.

실제 반영 위치: frontend/src/api.js `request`, `MESSAGES` / frontend/src/main.jsx `run`

내가 확인하고 수정한 내용: 403에서 `clearSession`이 호출되지 않는지 코드로 확인했고, 로그아웃 버튼이 메모리 세션과 함께 수정 중이던 입력(`reset()`)도 지우는지 확인했습니다. 코드는 수정 없이 반영했습니다.

검증 방법과 결과: `npm run build` 성공, 4173에서 UI 확인 (4장 그림 5, 그림 7). 401 뒤 "로그아웃 상태"로 바뀌고 403은 로그인 표시가 유지됨.

대표 사례 3 — 테스트 증거 자동 기록

도구 / 날짜: Claude Code / 2026-10-28

요청한 내용: "T01~T17을 curl로 수행하고 토큰을 가린 로그로 남길 것"

AI가 제안한 내용: `tools/manual_tests.sh` — 각 테스트의 요청·사용자·핵심 응답 헤더·본문을 기록하고 실패 뒤 재조회로 데이터 불변을 문자열 비교로 확인. 만료는 TTL=3 서버를 따로 띄워 수행.

실제 반영 위치: tools/manual_tests.sh, docs/evidence/*.log

내가 확인하고 수정한 내용: 로그에 토큰 원문이 남지 않는지 `grep eyJ`로 확인했습니다.

검증 방법과 결과: 45개 요청 블록이 모두 예상 HTTP/code와 일치.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
교수님 제공 시작 템플릿과 문서 (02_시작템플릿_및_제출양식: 공지, API명세, 체크리스트, 테스트시나리오, 로컬실행안내, check_api.py)	프로젝트 전체	템플릿 코드 유지, 명세의 상태 코드·클레임·처리 순서 기준
node-jsonwebtoken README — https://github.com/auth0/node-jsonwebtoken	backend/src/auth.js	`jwt.sign` 옵션(expiresIn, issuer, audience, algorithm), `jwt.verify` 옵션(algorithms, issuer, audience), `TokenExpiredError`
Express 5 Guide — https://expressjs.com/en/guide/error-handling.html	backend/src/app.js 이해	제공 코드의 공통 에러 핸들러 동작 이해
Vite Guide — https://vite.dev/guide/	frontend/vite.config.js 이해	preview 서버와 proxy 설정

6 이해도 확인과 회고

1. **요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?**

본문의 authorId를 그대로 저장하면 Bob이 `{"title":..., "authorId":1}`을 보내 Alice 이름으로 글을 쓰거나, 수정 요청에 authorId를 바꿔 넣어 작성자를 가로챌 수 있습니다. 로그인은 Bob으로 했는데 글은 Alice 것이 되는 셈입니다. 제 코드는 `POST /api/posts`에서 작성자를 `currentUserId(req)`가 토큰에서 검증한 ID로 정해 `store.create`에 넘기고, 본문에 authorId·userId 같은 허용되지 않은 필드가 있으면 `fields`가 400 BAD_INPUT으로 거부합니다(T12). 수정도 본문에서 작성자를 받지 않고 `store.update`는 title·content만 바꿉니다.

2. **수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?**

버튼은 브라우저 안의 JavaScript가 그리는 것이라 사용자가 개발자 도구나 curl로 `PUT /api/posts/5`를 직접 보낼 수 있습니다. 프론트의 `canEdit`는 편의 기능일 뿐 보안이 아닙니다. 실제 검사는 서버 `backend/src/auth.js`의 `assertOwner(userId, post.authorId)`가 하며, 토큰에서 검증한 사용자 ID와 DB의 작성자가 다르면 403 FORBIDDEN을 던져 `store.update`·`store.remove`에 도달하지 못하게 합니다. T07·T08에서 버튼 없이 Bob 토큰으로 직접 요청해 403과 데이터 불변을 확인했습니다.

3. **JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?**

decode는 토큰의 base64url 문자열을 풀어서 내용을 읽기만 합니다. 서명을 확인하지 않으므로 payload를 고쳐 다시 인코딩한 토큰도 똑같이 읽힙니다. verify는 header·payload를 서버 비밀 키로 다시 서명해 세 번째 조각과 비교하므로, 키를 모르는 사람이 내용을 바꾸면 서명이 맞지 않아 실패합니다. 제 `currentUserId`는 `jwt.verify`로 서명, 허용 알고리즘(HS256만), exp 만료, iss, aud를 검사하고, 추가로 exp 존재, sub가 양의 정수 문자열인지, 그 사용자가 실제로 있는지까지 확인합니다. 토큰 내용은 키 없이 누구나 읽을 수 있기 때문에 비밀번호나 해시는 넣지 않았습니다.

4. **로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?**

로그아웃은 브라우저 메모리의 `session` 변수를 비워 이 탭에서 더 이상 토큰을 보내지 않게 합니다. 서버는 아무것도 바뀌지 않습니다. 토큰 만료는 서버 쪽 판단입니다. exp가 지난 토큰은 서버가 401 TOKEN_EXPIRED로 거부하고, 프론트는 그 401을 받고 세션을 지웁니다. 새로고침은 JavaScript 메모리가 초기화되어 `session`이 빈 값으로 돌아가는 것이고, 글은 서버 DB에 있으니 그대로 보입니다. 이미 복사된 토큰은 로그아웃해도 서버에서 무효화되지 않으며 exp(기본 90초)가 지날 때까지 유효합니다. 그래서 TTL을 짧게 두고, 토큰을 localStorage에 남기지 않는 것입니다.

5. **가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.**

[본인이 겪은 내용으로 다시 쓰세요. 아래는 이번 작업에서 실제 있었던 일 두 가지입니다.] (1) 클레임 검사 스크립트를 처음 `tools/check_claims.mjs`에 두고 실행했더니 `ERR_MODULE_NOT_FOUND: Cannot find package 'dotenv'` 오류가 났습니다. 원인은 ES 모듈이 스크립트 파일 위치 기준으로 `node_modules`를 찾는데 `tools/`에는 없고 `backend/`에만 있기 때문이었습니다. 스크립트를 `backend/scripts/`로 옮기고 `backend` 폴더에서 실행해 해결했고, 10건 모두 기대한 응답이 나오는 것으로 재검증했습니다. (2) 이전 작업 세션에서 켜 둔 백엔드(BE-01까지만 구현된 상태)가 8000 포트를 계속 잡고 있어서 새 코드를 반영한 서버가 8000에서 시작하지 못했습니다. `netstat -ano`로 PID를 확인한 뒤, 테스트는 `PORT=8001`로 띄운 서버에서 진행하고 최종 UI 확인은 구 프로세스를 종료하고 8000에서 다시 했습니다. "코드를 고쳤는데 응답이 그대로"일 때 어떤 프로세스가 응답하는지 먼저 확인해야 한다는 것을 배웠습니다.

현재 남은 문제와 다음에 개선할 점: 없음. (의도된 제한: 메모리 보관 토큰이라 새로고침 시 재로그인 필요, 로그아웃이 서버 측 토큰 무효화는 아님)

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행 [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거

GitHub 확인: [x] 루트 README 작성 [ ] 보고서 PDF와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL과 커밋 SHA 제출